

Exercise 1: Introduction to Classes and Objects

Online Shopping Cart System

Overview

- **In-class exercise:** to be completed within the allocated lab time.
- **Duration:** 1 hour 15 minutes (75 minutes).
- **Strategy:** Pair Programming (two students collaborate on one computer).
- **Submission:** Submit the source code (.cpp file) via elearning before the deadline stated on the portal.

Pair Programming Rules

The Two Roles

 Driver	Types the code and controls the keyboard. Focus on writing clean, working code based on what you and your Navigator have discussed.
 Navigator	Reviews the code in real time, spots errors, suggests improvements, and guides the overall direction of the solution. The Navigator does not type.

Ground Rules

- **One computer per pair.** The Driver controls the keyboard. The Navigator must not type on a separate device.
- **Sit side by side.** Both members must be able to see the screen clearly at all times.
- **Switch roles at least once.** Both members must experience both roles during the session. You may switch after completing any task — the timing is flexible.
- **Stay engaged throughout.** The Navigator should speak up, not sit silently. If the Navigator is quiet, the lecturer will take note.

How the Lecturer Assesses Pair Programming

The lecturer will **walk around the class twice** during the session; once around the 25-minute mark and again around the 50-minute mark. During each round, the lecturer will observe your pair and record:

- **Whether both members are actively engaged** (asking questions, discussing, solving problems together).
- **Whether a role switch has taken place** (both members have held the keyboard at some point).

These observations are **recorded by the lecturer** and contribute **20 out of 100 points** to your exercise grade. You do not need to do anything, just collaborate genuinely and the marks will reflect what the lecturer sees.

Role Tracking Log

The Role Tracking Log for this exercise is completed **online via Google Sheet**, not on paper. The lecturer will share the link at the start of the lab session (in elearning).

What you need to do in the Google Sheet:

- Find your pair row (by Pair Number assigned by your lecturer).
- Fill in your names and Student IDs at the start of the lab.
- For each task, enter who is **Driver** and who is **Navigator**. Update this **as you work**, do not leave it until the end.
- Select ☒ **Yes** in the “Roles Switched?” column each time you swap roles.
- At the end of the lab, both members must select ☒ **Yes - both agree** in the Student Declaration column as confirmation.

The lecturer monitors the Google Sheet in real time during the lab. Your entries are visible to the lecturer as soon as you type them. This (together with direct observation) is how your pair programming marks are determined.

⚠ Do not edit another pair's row. Both members must appear as Driver for at least one task. A log where only one member's name appears as Driver throughout will result in deduction of marks for the Role Switching criterion.

No.	Task	Driver (Name / Student ID)	Navigator (Name / Student ID)	Roles Switched?	Lecturer Initials
1	Task 1: Class Declaration & Private Members			☐ Yes ☐ No	
2	Task 2: setItem() Mutator Function			☐ Yes ☐ No	
3	Task 3: Accessor Functions (Getters)			☐ Yes ☐ No	
4	Task 4: calculateTotal() & isEligibleForDiscount()			☐ Yes ☐ No	
5	Task 5: displayCart() & main()			☐ Yes ☐ No	

Academic Integrity & Penalties

- You may refer to lecture notes, textbooks, and online resources. However, **code that is copied or shared between pairs will be treated as plagiarism** and will result in **dismissal of the entire submission with no appeal**.
- **Programs that cannot compile** will receive a **50% penalty on the code score**.
- **Late submissions** will be penalised at **10% per 10 minutes** past the deadline.



Question: Online Shopping Cart System

Write a complete C++ program that simulates a simple **online shopping cart system**. The cart holds exactly **3 items**. Each item has a **name**, a **price per unit**, and a **quantity**. The program allows the user to enter item details, calculate the total cost, check whether a discount applies, and display a receipt.

Class Design: ShoppingCart

Before writing any code, study the class design below carefully. Declare the class named **ShoppingCart** following this exact structure:

Category	Member	Access
Data Members	<code>string itemNames[MAX_ITEMS]</code>	<code>private</code>
	<code>double itemPrices[MAX_ITEMS]</code>	<code>private</code>
	<code>int itemQty[MAX_ITEMS]</code>	<code>private</code>
Member Functions	<code>void setItem(int, string, double, int)</code>	<code>public</code>
	<code>string getName(int) const</code>	<code>public</code>
	<code>double getPrice(int) const</code>	<code>public</code>
	<code>int getQty(int) const</code>	<code>public</code>
	<code>double calculateTotal() const</code>	<code>public</code>
	<code>bool isEligibleForDiscount() const</code>	<code>public</code>
	<code>void displayCart() const</code>	<code>public</code>

Global constant: `const int MAX_ITEMS = 3;` ⇨ declared above the class definition. Use this constant in all array declarations and loops throughout the program.

Task 1: Class Declaration and Private Members

Declare the **ShoppingCart** class with the following **private** data members:

- `string itemNames[MAX_ITEMS]` - stores the name of each item.
- `double itemPrices[MAX_ITEMS]` - stores the price per unit of each item.
- `int itemQty[MAX_ITEMS]` - stores the quantity of each item.

Declare all **public** member functions as **prototypes** inside the class. Define each function **outside** the class using the scope resolution operator (`::`).

Task 2: setItem() Mutator Function

Write one mutator function named **setItem()**:

- `void setItem(int index, string name, double price, int qty)`

This function must:

- Store `name`, `price`, and `qty` into the arrays at position `index`.
- Validate inputs before storing:

- ✓ If `price ≤ 0`, display: "Invalid price. Price must be greater than RM0.00." and return without storing.
- ✓ If `qty ≤ 0`, display: "Invalid quantity. Quantity must be at least 1." and return without storing.

 **Tip for the Navigator:** Make sure the validation checks come **before** the array assignments. If either check fails, the function must return immediately and must **not** store anything.

Task 3: Accessor Functions (Getters)

Write the following **accessor functions**. All must be declared with `const` since they do not modify any data:

- i. `string getName(int index) const` - returns the name of the item at the given index.
- ii. `double getPrice(int index) const` - returns the price per unit of the item at the given index.
- iii. `int getQty(int index) const` - returns the quantity of the item at the given index.

Task 4: `calculateTotal()` and `isEligibleForDiscount()`

- i. `double calculateTotal() const` - calculates and returns the total cost of all items. Compute `price × qty` for each item and sum them. Loop `MAX_ITEMS` times.
- ii. `bool isEligibleForDiscount() const` - returns `true` if the total from `calculateTotal()` is RM50.00 or more, and `false` otherwise.

 **Tip for the Navigator:** In `isEligibleForDiscount()`, call `calculateTotal()` instead of rewriting the loop. Reusing an existing function is better practice and easier to maintain.

Task 5: `displayCart()` and `main()`

- i. `void displayCart() const` - displays a formatted receipt showing the name, unit price, quantity, and subtotal (`price × qty`) for each item; the total before discount; the discount amount (if eligible); and the final total after discount. Call `isEligibleForDiscount()` to decide whether to apply the 10% discount.

Write a **main()** function that:

- i. Declares a **ShoppingCart** object.
- ii. Uses a `for` loop to prompt the user for the name, price, and quantity of each item. Call `setItem()` for each item using the loop index (0, 1, 2).
- iii. Calls `isEligibleForDiscount()` and displays a message:
 - If eligible: "You qualify for a 10% discount!"
 - If not eligible: "No discount applied."
- iv. Calls `displayCart()` to display the final receipt.

 **Tip for the Navigator:** Check that `displayCart()` applies the discount by calling `isEligibleForDiscount()`, not by checking the total directly. Also verify that all three accessor functions (`getName()`, `getPrice()`, `getQty()`) are called somewhere in the program.

Important Notes

- Write the entire program in a **single source code file (.cpp)**.
- No constructors, destructors, or dynamic memory are required.

- **All member functions** declared in the class must be called at least once in `main()` or through other member functions.
- All function definitions must use the scope resolution operator (`::`).

Sample Program Output

Example 1: Total $\geq$ RM50 (Discount Applied):

```
Enter item 1 name: Notebook
Enter item 1 price: 12.00
Enter item 1 quantity: 2
Enter item 2 name: Pen
Enter item 2 price: 5.00
Enter item 2 quantity: 4
Enter item 3 name: Ruler
Enter item 3 price: 8.00
Enter item 3 quantity: 3

You qualify for a 10% discount!

===== Cart Receipt =====
Item           Price    Qty    Subtotal
-----
Notebook       12.00     2      24.00
Pen            5.00     4      20.00
Ruler          8.00     3      24.00
-----
Total before discount:           RM 68.00
Discount (10%):                 -RM 6.80
Final Total:                     RM 61.20
=====
```

Example 2: Total < RM50 (No Discount):

```
Enter item 1 name: Eraser
Enter item 1 price: 1.50
Enter item 1 quantity: 2
Enter item 2 name: Pencil
Enter item 2 price: 2.00
Enter item 2 quantity: 3
Enter item 3 name: Clip
Enter item 3 price: 3.00
Enter item 3 quantity: 2

No discount applied.

===== Cart Receipt =====
Item           Price    Qty    Subtotal
-----
Eraser         1.50     2       3.00
Pencil         2.00     3       6.00
Clip           3.00     2       6.00
-----
Total before discount:           RM 15.00
Discount (10%):                 -RM 0.00
Final Total:                     RM 15.00
=====
```

Example 3: Invalid Input:

```
Enter item 1 name: Notebook
```

```
Enter item 1 price: -5.00
Invalid price. Price must be greater than RM0.00.
```

Assessment

This exercise carries **3%** of the final course grade. The breakdown (out of 100 points) is as follows:

Criteria	Points
1. The Code	
a. Task 1 – Class declaration, private arrays, and access specifiers	15
b. Task 2 – setItem() mutator with input validation	15
c. Task 3 – Accessor functions (getName, getPrice, getQty)	10
d. Task 4 – calculateTotal() and isEligibleForDiscount()	20
e. Task 5 – displayCart() and main() function	20
2. Pair Programming Session (observed by lecturer during lab)	
a. Active collaboration: <i>Both members are actively engaged throughout: asking questions, discussing ideas, and solving problems together.</i>	10
b. Role switching: <i>Both members take turns as Driver and Navigator at least once. This is confirmed through the Role Tracking Log and lecturer observation.</i>	10
Total	100

How collaboration is assessed: The lecturer will walk around and observe each pair twice during the session. Observations are recorded independently by the lecturer. The Role Tracking Log in this document is used alongside the lecturer's observation notes to determine your pair programming marks.

Submission

- Submit your source code (.cpp file only) via **e-learning** before the deadline stated on the portal.
- Submissions via email, Telegram, Google Drive, or any other platform will **not be accepted**.
- The source code must be submitted **during class time**. Check e-learning for the exact deadline.

FAQs

1. **Who will be my partner?**
Choose your own partner. You should have arranged this before the session begins.
2. **Can I pair up with someone from a different section?**
No. You must work with someone from the same section.

3. **Can I do the exercise alone?**

Only if the class has an odd number of students and you have obtained prior permission from the lecturer.

4. **Do we need to switch roles?**

Yes. Both members must experience both the Driver and Navigator roles at least once. No single member should control the keyboard for the entire session.

5. **Can we both type on our own laptops?**

No. Only one device per pair is allowed. Using a second device for coding defeats the purpose of pair programming.

6. **What if my partner is not contributing?**

Inform the lecturer during the session, do not wait until after class. This will be reflected in the pair programming marks.

7. **How does the lecturer know if we switched roles?**

The lecturer monitors the Role Tracking Google Sheet in real time during the lab and also walks around to observe each pair directly twice. Both sources, what you enter in the sheet and what the lecturer sees in person are used together to determine your marks.

8. **Can we ask the lecturer questions during the session?**

Yes. Try to discuss and solve problems with your partner first, but you are always welcome to ask for clarification.

9. **Will our code and collaboration marks be separated?**

Yes. Code accounts for 80 points (marked after submission) and pair programming accounts for 20 points (observed during the lab). Both contribute to your final 100-point score.